

TECHNICAL WHITE PAPER

Bromure Web

A disposable-VM browsing architecture for the enterprise:
how we contain the web, see what employees do on it, and
force every corporate session through a managed
boundary.

Contents

1. Executive summary
2. Threat model: what the corporate browser is up against
3. The Bromure Web isolation architecture
4. Why this design is safe — five containment claims, defended
5. The visibility problem, and what enterprise management solves
6. Enforcement: making Bromure the only way in (Google Workspace, Entra, Okta)
7. Privacy, data residency, and the dual-use line
8. Operational model and rollout
9. Appendix A — Cryptographic specifics
10. Appendix B — Comparison with adjacent categories

1. Executive summary

Bromure Web turns the browser — the single largest unmanaged attack surface on a corporate endpoint — into a disposable, fully observable, fully controllable resource. Every browsing session runs inside its own short-lived Linux virtual machine on the employee's Mac. When the window closes, the VM and everything it touched are destroyed.

Built on Apple's `Virtualization.framework`, Bromure pre-warms a pool of VMs so a new session opens in under a second. Inside that VM, Chromium runs against a clean OS image, isolated from the host's files, keychain, network stack, and other browser sessions. The user sees a normal browser; what's actually running is a one-shot OS.

THE THREE CLAIMS THIS PAPER DEFENDS

- **Safety.** A successful browser exploit cannot read host files, persist on the device, or laterally attack the corporate network. The blast radius ends at the VM boundary, and the VM is destroyed on close.
- **Visibility.** For enterprise-managed sessions, every HTTP request, response, and form submission is recorded against a verified user identity and shipped to the customer's tenant — without the user being able to tamper with capture or disable it locally.
- **Enforcement.** Identity-provider conditional-access rules (Google Workspace, Microsoft Entra ID, Okta) can be configured to admit only requests presenting a Bromure-issued mTLS client certificate. That certificate exists exclusively inside managed Bromure VMs. Other browsers cannot pass the check.

The rest of the paper walks each claim back to the implementation: hypervisor, copy-on-write disks, virtio sockets, signed managed-profile manifests, mTLS leaf certificates, and the integration surface with enterprise identity providers.

2. Threat model: what the corporate browser is up against

The browser is the only piece of software on a corporate laptop that: loads untrusted code from arbitrary third parties every minute of the workday; holds long-lived session tokens for the company's most sensitive SaaS; can be re-targeted by a phishing link in any chat message; and runs with the user's own privileges on the host OS.

The threats Bromure Web is designed to defeat are:

Threat	Why the conventional browser fails	How Bromure changes the equation
Browser zero-day or n-day RCE	A renderer or GPU process compromise can pivot to the host: read <code>~/Documents</code> , the keychain, SSH keys, dotfiles.	Renderer runs inside a Linux guest under a hardware hypervisor. There is no <code>~/Documents</code> in the guest; the host is on the other side of a Type-1 boundary.
Malicious extension / supply-chain extension update	Persists across sessions, reads cookies for every site, exfiltrates indefinitely.	Extension set is server-controlled per managed profile. The disk it lives on is destroyed at window close. Persistence has no surface.
Drive-by malware / drive-by data theft	Dropped binary executes, harvests creds, beacons to C2.	Dropped binary lands in an ephemeral guest with no host filesystem reachable and no inbound vsock to abuse. The guest is shredded on close.
Phishing leading to credential paste	Credentials end up at the attacker's IdP-impersonation site, then the attacker logs into the real SaaS from a residential VPN.	The IdP requires the Bromure mTLS leaf cert as a precondition for finishing the login. The attacker's residential VPN doesn't have one. The credential is useless on its own.
Insider exfiltration via webmail or paste	Endpoint EDR sees a TLS connection but cannot read the body. Sensitive POSTs are invisible.	The managed profile's trace policy records request body and form fields in-VM, before TLS, and ships them to the customer tenant over mTLS.
Cross-session contamination	Two tabs in the same browser share cookies, storage, and the same renderer pool.	Each session is a separate VM, separate IP, separate cookie jar, separate kernel.

Out of scope for this paper: physical attacks on the Mac itself, compromise of macOS below the user account, and supply-chain attacks on Apple's Virtualization.framework. We rely on Apple's hypervisor as the trust anchor in the same way every macOS security product does.

3. The Bromure Web isolation architecture

3.1 One VM per session

A Bromure Web "session" is one browser window. Behind that window sits a dedicated Linux VM, booted from a sealed Bromure-built base image cloned via APFS copy-on-write — so cloning a 4 GB image is an O(1) metadata operation, not a 4 GB copy. The VM has its own kernel, its own filesystem, its own MAC address, and its own network namespace.

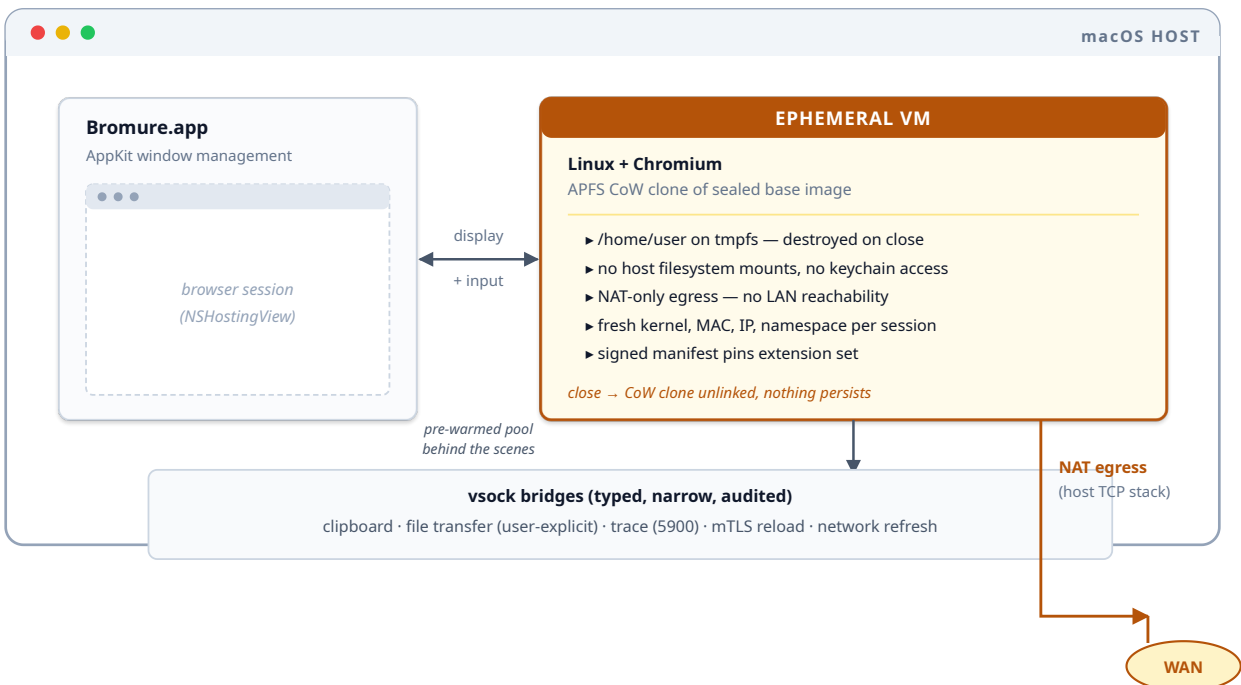


Figure 1 — Per-session VM with explicit, narrow channels to the host.

When the user closes the window, the VM is shut down, its CoW clone is unlinked from the APFS volume, and the only durable artifact that remains is whatever the trace pipeline (§5) has already shipped to the customer tenant — not anything on the device.

3.2 The pool and the cold start problem

A throwaway VM is useless if it takes 30 seconds to start. Bromure keeps a small pool of pre-booted "warm" VMs sitting on the login screen of the in-guest Chromium. When the user opens a new window, the pool hands one over and immediately starts warming a replacement in the background. Perceived cold-start is sub-second.

3.3 Channels between host and guest

Linux guests on Apple Silicon talk to the host over **virtio sockets** — a typed, point-to-point channel, not a network. A guest cannot accidentally send a packet to the corporate LAN by binding to the wrong interface; it can only speak the protocols we expose. The full list is small and audited:

Channel	Direction	Purpose
Clipboard bridge	both	User-initiated copy/paste. Off by default for managed profiles that forbid it.
File-transfer bridge	both	User-explicit drag/drop. No filesystem mount, no implicit access.
Trace bridge (port 5900)	guest → host	HTTP trace events from in-guest CDP listener.
mTLS reload bridge	host → guest	Push renewed client certificate without restarting the VM.
Network-refresh bridge	host → guest	Notify the guest that the host's upstream link changed (Wi-Fi switch, VPN up/down).

This is a deliberately small surface. Everything else — DNS, HTTP, IPC to host services — is unavailable to the guest. The guest cannot reach `localhost` on the host. It cannot enumerate the host's LAN. It cannot probe the user's network printer.

3.4 Guest network egress

Guest traffic is NAT'd through the host's TCP stack to the WAN. The host can optionally interpose a network filter or push the guest through an enterprise IKEv2 or WireGuard tunnel so all browsing egresses from a corporate gateway. For an admin, this gives a single place to apply DNS filtering, URL allow/deny, and IP allow-listing for sensitive SaaS — exactly where the regulatory framework wants it: at the boundary, not on the endpoint.

4. Why this design is safe — five containment claims, defended

Each subsection states a claim, names the property in the implementation that establishes it, and points at what would have to be true for the claim to fail.

4.1 An RCE in the renderer cannot read host files

Chromium runs in a Linux guest. The guest's view of "the filesystem" is the CoW-cloned APFS image; it does not include the user's home directory, the keychain, the host's TLS store, or anything the user might lose sleep over. There is no `9p` or `VirtioFS` mount exposing host files. File transfers happen by an explicit user action that the host approves; there is no implicit shared mount.

For an attacker to reach a host file from inside the guest, they would need a chained exploit through Apple's `Virtualization.framework` — the same hardware-backed boundary that protects every other hypervisor product on Apple Silicon.

4.2 Nothing persists across sessions

Each VM is a fresh APFS CoW clone of the sealed base image. The clone is unlinked when the window closes. There is no per-user scratch volume. There is no cookie jar that survives. A malicious extension installed by exploit at `t=0` is gone at `t=close`; the next session starts from the same pristine image as the first.

Implication for IR. "Wipe and re-image" is not an incident-response step in Bromure — it is the steady state. Every close is a wipe. The mean time between fresh images, per browser window, is the length of one browsing task.

4.3 One session cannot see another

Two windows are two VMs. They share no kernel, no memory, no storage, no cookies. A cookie stolen in window A is not visible in window B, even for the same user, even for the same site. This is qualitatively stronger than Chromium's "profile" or "container" features, which are software boundaries inside one process tree.

4.4 The guest cannot pivot onto the corporate LAN

The guest has no L2 reachability to the host's LAN. Traffic is NAT'd out of the host's TCP stack. A guest cannot ARP the user's printer, scan the office /24, or open an SMB connection to a file server. The path off the host is a forwarded socket, not a bridged interface.

4.5 The supply chain for what the user runs is signed and pinned

Managed profiles arrive as **signed manifests** from the control plane. The manifest enumerates settings, extensions, and asset SHA-256s. Bromure refuses to load a profile whose signature does not verify against the org's signing key. Extension bundles are content-addressed and cached; a tampered cached blob is rejected at load time. The user has no UI to "trust this extension anyway" for managed profiles — the surface for social-engineering an end user into running malicious code is closed.

SUMMARY OF THE SAFETY STORY

- The renderer runs on the wrong side of a hardware hypervisor.
- The disk it writes to dies with the window.
- The network it speaks on is NAT-only, not LAN-bridged.
- The extensions it loads are signature-pinned by the admin, not the user.
- The IPC channels it has to the host are typed, narrow, and audited.

5. The visibility problem, and what enterprise management solves

5.1 What endpoint security cannot see

Standard endpoint EDRs see process trees, file writes, and the outside of TLS connections. They cannot read encrypted HTTP bodies. They cannot tell you which fields of a form a user submitted. They cannot link a specific HTTP POST to a specific employee identity in a way that survives load balancers or shared egress.

For an industry that increasingly does its work entirely inside the browser, this means the most important workplace activity — the document uploaded to a personal Gmail, the prompt pasted into a third-party LLM, the credential typed into a phishing replica of the IdP — is the part the security team is most blind to.

5.2 Where Bromure captures, and why that's the right layer

Bromure observes traffic **inside the browser, before TLS encrypts it**. The capture point is the Chrome DevTools Protocol's Network domain running on the in-guest Chromium. Every request, response, header, body, and form-field submission is surfaced as a typed event. The event is shipped over a guest → host vssock channel (port 5900) to the host's Trace Bridge, then handed to the Cloud Trace Uploader for batched delivery to the customer tenant.

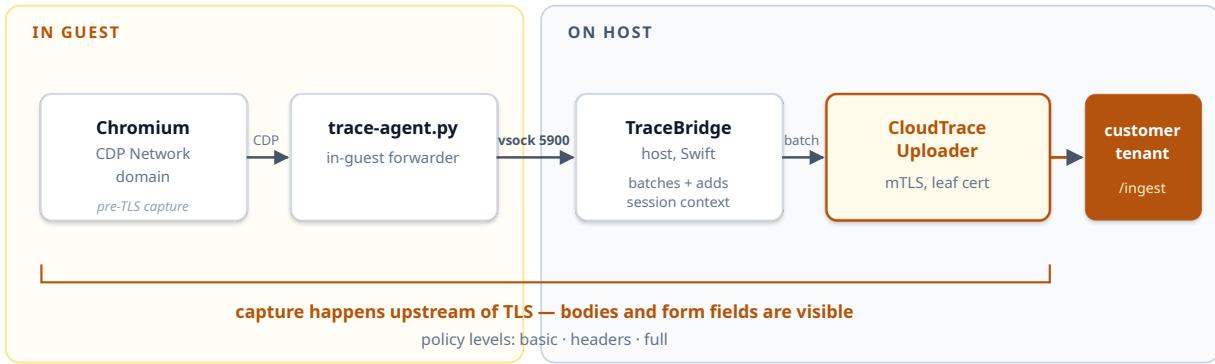


Figure 2 — In-browser capture, host-side batching, mTLS shipping. The capture point is upstream of TLS.

5.3 Three policy levels, set by the admin

The amount of detail captured is a server-controlled policy attached to the managed profile. It cannot be lowered by the end user. The three levels:

BASIC

URL, HTTP method, status code, timing. No headers. No bodies. Suitable for audit and incident scoping where content sensitivity is high.

HEADERS

Basic, plus request and response headers. Distinguishes API calls from page loads, captures content types, surfaces auth scheme metadata without recording the credential itself.

FULL

Headers, plus request body, response body, and explicit form-field snapshots. Captures the document pasted into a chat, the prompt typed into an LLM, the field filled in a phishing form.

OFF

No upload. The profile's behavior collapses to the unmanaged Bromure default: a session-local trace inspector that lives and dies with the window.

5.4 Why the user can't turn it off

The trace policy is delivered inside a signed manifest from the control plane. The host enforces it; the in-window UI for "stop recording" is disabled and read-only for managed profiles where the level is non-off. The end user cannot edit the profile, cannot rotate the cert, cannot swap the endpoint. The local trace store is bypassed entirely — events go straight to the cloud uploader and never touch local SQLite.

5.5 Why the data on the wire is trustworthy

Events are shipped over **mTLS**, authenticated by the managed profile's leaf certificate. The server identifies the install by the cert's subject CN; bearer tokens never ride with trace data. Each batch is

denormalized with the install ID, org slug, and the verified user email recorded at enrollment time, so every captured request is searchable per-employee in the customer's tenant.

For a VC reading this: the "visibility" product is the part that creates lock-in. Once an enterprise wires its conditional-access rules to require the Bromure leaf cert, every sensitive SaaS interaction by every employee flows through a controlled, observable channel they own — and replacing Bromure means rebuilding that control plane.

6. Enforcement: making Bromure the only way in

Visibility into a managed browser is worthless if employees can bypass the managed browser by opening Safari, Chrome, or a personal Edge instance for the same SaaS. The enforcement story is how Bromure becomes the only path that succeeds — not by software on the endpoint, but by a policy at the identity provider.

6.1 The enrollment and certificate flow

When a user is added to an org in the Bromure control plane, they receive a six-word enrollment code (e.g. `acid-aloe-arson-bench-cat-drum`). On their Mac, they redeem it:

```
$ bromure enroll acid-aloe-arson-bench-cat-drum
Enrolling against https://bromure.io/api as Renaud-MBP...
Enrolled. Received 2 managed profile(s):
- Sales SaaS v7 mTLS=yes assets=3
- Finance v3 mTLS=yes assets=2
```

Behind that single command:

1. An X25519 install keypair is generated and the private half is stored in the macOS Keychain with `WhenUnlockedThisDeviceOnly`.
2. The install is registered with the control plane. The server returns an install token and the set of profiles assigned to the user (directly or through groups).
3. For each managed profile that has `mTLS.enabled`, the control plane's per-org CA issues a leaf certificate whose subject CN ties the cert to a specific (install, profile, user) tuple. The private key is generated on the device and never leaves it.
4. Inside every VM that boots for that profile, the leaf cert + key are imported into Chromium's NSS DB at startup. The browser presents the cert during TLS handshakes whenever a server requests one.

6.2 The IdP conditional-access rule

The admin then configures their identity provider — Google Workspace, Microsoft Entra ID, or Okta — to require a valid Bromure-issued client certificate for the corporate SaaS apps they care about. This is a single rule, configured once per app, and it is the linchpin of the entire enforcement model.

Identity provider	Mechanism	Effect
Google Workspace	Context-Aware Access policy that requires a client certificate issued by the org's uploaded Bromure CA trust chain.	Login to Workspace apps (Gmail, Drive, Sheets) and any SAML/OIDC SaaS fronted by Workspace SSO fails for browsers that don't present the cert.
Microsoft Entra ID	Conditional Access policy with "Require approved client app" / certificate-based authentication, using the Bromure CA as a trusted issuer.	Access to Microsoft 365, and any SaaS using Entra as IdP, is gated on cert presentation.
Okta	Okta Device Assurance with client certificate, anchored on the Bromure CA.	Any application behind Okta SSO inherits the constraint.

6.3 What this looks like to users and attackers

LEGITIMATE EMPLOYEE

Clicks the SaaS link from Slack. Bromure intercepts and routes the URL into a managed-profile VM (or pops the OS handler that directs `https` URLs to Bromure). The in-VM browser presents the leaf cert during the IdP handshake. IdP is happy. Login proceeds as normal. The user never sees a cert dialog.

PHISHED CREDENTIAL

Attacker has the password from a typo-squatted login page. They try to sign in from a residential proxy in Chrome on Windows. The IdP demands a client certificate. Their browser has none. The login is rejected before any session token is issued. The stolen password is worth nothing.

INSIDER ON PERSONAL BROWSER

Employee opens Safari to bypass logging and download a sensitive document to a personal Drive. Workspace rejects the login because Safari doesn't have a Bromure cert. To download the document they must use Bromure, where the trace pipeline records the action.

LOST LAPTOP

Admin unenrolls the install in the control plane. The next profile sync removes the leaf cert (and revokes it server-side). The IdP rejects any further attempts to authenticate from that device. The user keeps the Mac; the org keeps its data.

6.4 Why this is stronger than "managed browser" or "device-bound token"

Existing approaches require either deploying a fork of Chrome with proprietary management hooks (a heavy maintenance burden and a parallel browser the user has to remember to use), or relying on a

device-bound bearer token that lives in the OS keychain and can be read by any process on that user account, including malware. The Bromure approach inverts both:

- **The cert lives inside the VM**, not on the host. Malware on the host's user account cannot read it; the host doesn't have it.
- **The cert is profile-scoped**. The Finance VM has the Finance cert; the Sales VM has the Sales cert. Cross-profile pivoting is structurally impossible.
- **Renewal is silent**. The host's `MTLSReloadBridge` pushes new material into a running VM without restarting it; users never see expiration prompts.

7. Privacy, data residency, and the dual-use line

Bromure Web is, deliberately, a tool that lets administrators see more than employees might expect a corporate browser to surface. We treat that as a feature only inside a strict frame:

- **Managed profiles only**. Cloud trace upload is bound to managed profiles delivered by the org's control plane. The unmanaged Bromure user has no upload pipeline, by construction.
- **Server-controlled endpoint**. Each org points the trace stream at a tenant they own (or, for hosted customers, at `analytics.bromure.io/ingest` under a tenant-bound DPA). The endpoint is part of the signed manifest, so it cannot be silently redirected.
- **Visible indicator**. Sessions whose trace level is non-off are shown with a recording indicator in the window chrome. The user knows when the session is being captured. This is a non-negotiable UX requirement.
- **Profile-bound capture**. Trace recording only happens when a session is opened under a managed profile that mandates it. A user opening a personal browsing session under their own (unmanaged) profile is not captured.

Data residency is a function of where the customer points the ingest endpoint. For Bromure-hosted customers, regional ingest clusters are part of the SLA. For self-hosted customers, traces never leave the customer's cloud account.

8. Operational model and rollout

8.1 Day-zero deployment

1. Admin signs up at `bromure.io`, creates an org, uploads (or generates) the org's signing key and the CA the IdP will trust.
2. Admin defines one or more managed profiles: which extensions, which network policy, which trace level, which SaaS endpoints they cover.
3. Admin mints enrollment codes (one per user, or one per group with capacity).

4. End users install Bromure (or have it installed by MDM) and run `bromure enroll` once.
5. Admin configures the IdP rule to require the Bromure CA on the SaaS apps in scope. After this step, those apps are reachable only from Bromure-managed sessions.

8.2 Steady-state operations

- Profile updates (new extension, tighter URL allow-list, raised trace level) propagate at next sync; users do not need to restart.
- mTLS certs renew silently via the host → guest reload channel.
- Traces appear in the customer tenant in near real time, searchable by user, profile, URL, and form field.
- Off-boarding is a single unenroll in the control plane; the cert revokes, the IdP rejects, the device loses access.

8.3 What breaks well

The system is built to degrade visibly rather than silently. If the control plane is unreachable, existing sessions keep working with the last signed manifest; new enrollments are blocked. If the trace endpoint is unreachable, the host uploader buffers and retries — bounded by a queue cap that drops oldest events first rather than newest, so the recent activity around an incident is the part most likely to survive.

Appendix A — Cryptographic specifics

Purpose	Primitive	Note
Install identity	X25519 keypair	Generated at enroll, private half in macOS Keychain (device-only).
Profile manifest signature	Ed25519 over a canonicalized JSON serialization	Org signing key; public key shipped with the manifest, verified against the org record at the control plane.
Sealed payload delivery	libsodium-style sealed box to the install pubkey	Lets the server pre-encrypt assets for a specific install without holding their key material.
mTLS leaf cert	ECDSA P-256 over org CA	Subject CN encodes (orgSlug, installId, profileId); short validity; renewed silently.
Trace upload transport	TLS 1.3 with mutual auth	Server identifies install by client cert; no bearer token on the wire.
Asset integrity	SHA-256 in manifest	Verified on load; mismatched assets are refused.

Appendix B — Comparison with adjacent categories

	Endpoint EDR	Enterprise browser (Island, Talon)	Remote browser isolation (RBI)	Bromure Web
Where the browser runs	Host OS	Host OS (custom Chromium fork)	Vendor cloud (pixel-stream)	Ephemeral VM on the user's Mac
Reads HTTPS bodies	No	Yes (in-process)	Yes	Yes (in-VM CDP, pre-TLS)
Persistence after window close	Yes (host disk)	Yes (host disk)	Vendor-managed	None
Sandbox boundary	Process	Process	Network	Hardware hypervisor
Enforcement model	Endpoint policy	Forked browser binary + MDM	DNS / proxy redirect	IdP-bound client certificate inside the VM

	Endpoint EDR	Enterprise browser (Island, Talon)	Remote browser isolation (RBI)	Bromure Web
Latency cost	None	None	High (cloud pixel-stream)	~Sub-second (local VM)
Works offline	Yes	Yes	No	Yes (host alone is enough)

This document describes the Bromure Web architecture as of May 2026. Implementation details — file paths, vsock port numbers, and the shape of in-VM agents — are accurate to the current release but may evolve. The architectural claims about safety, visibility, and enforcement are stable.